

Final Core Feature Development and Implementation Report

Name: Louis Francis Lim

Project: GymTrack

GitHub Repository: <https://github.com/louisflim/gymtrack>

1. Core Feature Overview

1.1 Purpose

The QR Code-Based Attendance feature is the mechanism GymTrack uses to record when a member physically checks in and checks out of a gym, without requiring manual sign-in sheets, biometric hardware, or staff typing member names. It answers three operational needs of a small-to-medium gym:

- **Fast, low-friction verification at the front desk** — staff scan a member's phone screen instead of searching a member list.
- **Enforcement of business rules at the point of entry** — a member with an expired or unpaid subscription cannot be checked in, which ties attendance directly to the payments/membership subsystem.
- **A per-branch onboarding path** — a dedicated “Gym QR” lets a newly registered member self-link their account to the correct gym branch on their first visit, before staff need to do anything.

Architecturally, this feature is the integration point between four other subsystems already present in GymTrack: authentication/roles (who is allowed to scan), membership/subscriptions (whether a scan is allowed to succeed), gyms (multi-branch scoping), and the member/staff dashboards (where the QR and the resulting logs are displayed).

2. Functional Requirements

- **FR-1:** The system shall generate a unique, scannable QR code for every MEMBER account, encoding that member's internal user ID ("GYMTRACK|USER|<id>").
- **FR-2:** The system shall generate a unique QR code per gym branch, encoding the gym's ID ("GYMTRACK|GYM|<id>"), for use in self-service gym enrollment.
- **FR-3:** The system shall refuse to issue a member QR code unless the member has an active or expiring-soon subscription plan.
- **FR-4:** The system shall restrict attendance scanning (/api/attendance/scan) to authenticated users with the STAFF or ADMIN role.
- **FR-5:** The system shall restrict gym-QR self-scanning (/api/attendance/scan-gym) to authenticated users with the MEMBER role.
- **FR-6:** The system shall reject an attendance scan if the scanned member does not belong to the scanning staff member's gym.

- **FR-7:** The system shall reject a check-in attempt if the member's account is deactivated or their membership does not allow attendance (expired/unpaid), while still allowing a pending check-out to close out.
- **FR-8:** The system shall automatically determine whether a scan represents a CHECK_IN or CHECK_OUT based on whether the member has an open (un-checked-out) attendance log.
- **FR-9:** The system shall persist every check-in and check-out event with a timestamp, associated to the member's user account.
- **FR-10:** The system shall allow members to view their own attendance history and allow staff/admin to view and filter their gym's attendance history.

3. Core Feature Workflow

3.1 Two Complementary Flows

The feature implements two related but distinct workflows that share the same QR technology (ZXing) and payload format ("GYMTRACK|<TYPE>|<id>"):

Flow A — Gym Enrollment (Member scans a Gym QR)

A gym displays a static QR code (printed poster or screen) that encodes its own ID. A member who has just registered, but is not yet linked to a specific branch, opens the in-app scanner and scans this poster. The backend links the member's account to that gym. This is a self-service, one-time action per member (subsequent visits use Flow B).

Flow B — Attendance Check-In / Check-Out (Staff scans a Member QR)

Each member has a personal, permanent QR code (visible under “My QR Code” in their dashboard) that encodes only their user ID. On every visit, staff open the scanner on the front-desk device/app and scan the member's QR. The backend inspects the member's latest attendance log: if there is an open session (checked in, not yet checked out), the scan is treated as a CHECK_OUT; otherwise it is a CHECK_IN, subject to the member's subscription being valid.

3.2 End-to-End Narrative

1. Member purchases/activates a subscription plan (handled by the separate membership/payments feature).
2. Member opens “My QR Code”; the backend issues a QR only if their subscription currently allows attendance.
3. On their first visit, the member scans the gym's QR to link their account to that branch (Flow A).
4. On every visit thereafter, staff scan the member's personal QR (Flow B). The system automatically toggles between check-in and check-out.
5. Both members and staff/admin can review attendance history — members see only their own logs; staff/admin see their gym's logs, with search/date filtering.

4. Sequence of Actions

The table below traces each user-facing action to its system response, backend processing, database operation, and expected output.

#	User Action	System Response	Backend Processing	Database Operation	Expected Output
1	Member logs in and opens “My QR Code” screen (web or mobile).	Client calls GET /api/qr/me.	QrController.myQr() → AttendanceService.generateQrForUser(): looks up user, verifies role == MEMBER, verifies user is enrolled at a gym, fetches latest Membership and checks MembershipStatusUtil.allowsAttendance().	SELECT on users, memberships (latest by user, ordered by date) — read-only.	200 OK with { qrData: "GYMTRACK USER <id>", qrImageBase64 }. Frontend renders the PNG.
2	Member is denied a QR (no active plan).	Client calls GET /api/qr/me.	Same method throws IllegalArgumentException ("Purchase an active subscription...") when membership status is EXPIRED/NONE.	Read-only — no write occurs.	400 Bad Request with an explanatory message; UI shows a prompt to renew/purchase a plan.
3	Staff/Admin opens the gym's dashboard and displays the “Gym QR” (a poster/kiosk QR unique to that branch).	Client calls GET /api/qr/gym.	QrController.gymQr() → AttendanceService.generateGymQrForStaff(): GymGuard.requireStaffGym() confirms caller role is STAFF/ADMIN and is linked to a gym.	SELECT on users, gyms.	200 OK with { qrData: "GYMTRACK GYM <id>", qrImageBase64 }.
4	A new member scans the Gym QR (self-service enrollment/first visit) using the in-app camera scanner.	Client POSTs { qrData } to /api/attendance/scan-gym.	AttendanceService.scanGymQrAsMember(): validates caller role == MEMBER and account active; parses gym ID from payload; links member to that gym on first scan.	UPDATE users (set gym_id) as needed; SELECT gyms.	200 OK with MemberGymScanResponse (action, gymName, membershipStatus, nextStep). 400/403 on invalid QR, inactive account, or role mismatch.
5	Staff opens the “Scan Attendance” camera view and scans a member's personal QR.	Client POSTs { qrData } to /api/attendance/scan.	AttendanceService.scanAttendance(): GymGuard.requireStaffGym() restricts callers to STAFF/ADMIN; parses member ID; validates member is active, enrolled at the scanning gym, and membership allows attendance (unless this is a check-out); determines CHECK_IN vs CHECK_OUT from the last open log.	SELECT latest attendance_logs row for member; INSERT a new row (check-in) or UPDATE the open row's check_out_time (check-out); conditionally UPDATE users.first_check_in_completed.	200 OK with AttendanceScanResponse (action, member name, plan, status, timestamp, gym name, message). 400/403 on invalid QR, wrong gym, inactive account, or expired membership.
6	Member views their own attendance history.	Client calls GET /api/attendance/me.	AttendanceService.getMyAttendanceLogs(): resolves the caller and fetches their logs, most recent first.	SELECT on attendance_logs WHERE user_id = ? ORDER BY check_in_time DESC.	200 OK with a list of AttendanceLogResponse entries rendered in the member dashboard.

#	User Action	System Response	Backend Processing	Database Operation	Expected Output
7	Staff/Admin views the gym's attendance log (optionally filtered by member name or date).	Client calls GET /api/attendance/gym?search=&date=.	AttendanceService.getGymAttendanceLogs(): GymGuard restricts to STAFF/ADMIN; applies optional search/date filters scoped to the caller's gym.	SELECT on attendance_logs joined to users, filtered by gym_id (+ optional name/date predicates).	200 OK with the filtered log list rendered as a table in the admin dashboard.

5. Implementation Explanation

5.1 Major Source-Code Components

Component / File	Type	Responsibility
<code>QrController.java</code>	REST Controller	Exposes GET /api/qr/me and GET /api/qr/gym; delegates to AttendanceService and maps SecurityException → 403, IllegalArgumentException → 400.
<code>AttendanceController.java</code>	REST Controller	Exposes POST /api/attendance/scan-gym, POST /api/attendance/scan, GET /api/attendance/me, GET /api/attendance/gym; same error-mapping pattern.
<code>AttendanceService.java</code>	Service (business logic)	Core of the feature. Builds QR payloads, encodes QR images via ZXing, validates roles/gym membership/subscription status, determines check-in vs check-out, and persists AttendanceLog records.
<code>GymGuard.java</code>	Utility / authorization guard	requireStaffGym() and requireAdminGym() centralize the STAFF/ADMIN role check used by every gym-scoped operation, including QR generation and scanning.
<code>MembershipStatusUtil.java</code>	Utility	Computes a membership's status (ACTIVE / EXPIRING_SOON / EXPIRED / NONE) from its end date and exposes allowsAttendance() used to gate check-in.
<code>AttendanceLog.java</code> (entity)	JPA Entity	Maps to the attendance_logs table; stores user, check_in_time, check_out_time. @PrePersist defaults check_in_time to now().
<code>AttendanceLogRepository.java</code>	Spring Data Repository	Query methods such as findTopByUserIdOrderByCheckInTimeDesc() (to detect an open session) and gym-scoped/filterable log queries.
<code>QrCodeResponse / ScanRequest / AttendanceScanResponse / MemberGymScanResponse / AttendanceLogResponse</code>	DTOs	Define the JSON request/response shapes exchanged with the frontend and mobile clients.
<code>SecurityConfig.java</code> + <code>JwtAuthFilter</code>	Security configuration	Requires a valid JWT for all endpoints except /api/auth/** and the payment webhook; role-specific restriction is then enforced in the service layer via GymGuard.
<code>web/src/features/attendance/api.js</code>	Frontend API client (React)	Wraps the REST endpoints (fetchMyQrCode, fetchGymQrCode, scanGymQr, scanAttendance, fetchMyAttendanceLogs, fetchGymAttendanceLogs) using axios.

Component / File	Type	Responsibility
<code>web/src/features/attendance/QrScanner.jsx</code>	React component	Opens the device camera via @zxing/browser's BrowserMultiFormatReader, decodes a QR in real time, and returns the raw string to the caller; stops the video stream on success/close.
<code>MemberQrCard / GymQrCard / AttendanceScannerCard / AttendanceLogTable / AttendanceHistory (.jsx)</code>	React components	Render the member's personal QR, the gym's QR, the staff scan trigger, and the resulting attendance tables/history.
<code>mobile/.../feature/attendance/AttendanceRepository.kt</code>	Android repository	Calls the same REST endpoints from the native app via Retrofit (ApiService.kt).
<code>mobile/.../ui/.../QrScannerDialogFragment.kt</code>	Android UI (Fragment)	Hosts the camera scan UI using com.journeyapps:zxing-android-embedded, feeding the decoded string into AttendanceRepository.

5.2 QR Code Generation (Server-Side)

QR images are generated in `AttendanceService.generateQrBase64()`, using Google's ZXing library (`com.google.zxing:core` and `:javase`, declared in `backend/pom.xml`):

```
private String generateQrBase64(String payload) {
    QRCodeWriter qrCodeWriter = new QRCodeWriter();
    BitMatrix bitMatrix = qrCodeWriter.encode(payload, BarcodeFormat.QR_CODE, 320, 320);
    ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
    MatrixToImageWriter.writeToStream(bitMatrix, "PNG", outputStream);
    return Base64.getEncoder().encodeToString(outputStream.toByteArray());
}
```

The payload itself is a plain, deterministic string built from the entity's primary key — not a rotating token:

```
private String buildMemberQrPayload(Long userId) { return "GYMTRACK|USER|" + userId; }
private String buildGymQrPayload(Long gymId) { return "GYMTRACK|GYM|" + gymId; }
```

Because the payload never changes for a given user/gym ID, the resulting QR image is stable across calls — the member does not need to refresh or regenerate it between visits.

5.3 QR Code Scanning (Client-Side)

Decoding happens entirely on the client, using platform-appropriate ZXing bindings, and only the resulting text string is sent to the backend:

- **Web:** @zxing/browser's BrowserMultiFormatReader streams the device camera into a <video> element (`web/src/features/attendance/QrScanner.jsx`) and calls back with the decoded text.
- **Mobile (Android):** `com.journeyapps:zxing-android-embedded` provides a ready-made scanning UI, invoked from `QrScannerDialogFragment.kt`.

Neither client performs any authorization logic — all validation happens server-side once the decoded string reaches `/api/attendance/scan` or `/api/attendance/scan-gym`.

5.4 Validation and Error-Handling Mechanisms

5.4.1 Role-Based Authorization

Spring Security's filter chain (`SecurityConfig.java`) only enforces that a request is authenticated (valid JWT); it does not encode per-role URL rules. Role restriction is instead enforced explicitly in the service layer via `GymGuard`:

```
public static Gym requireStaffGym(User user) {    if (user.getRole() != Role.ADMIN &&
user.getRole() != Role.STAFF) {        throw new SecurityException("Only staff or admins can
perform this action.");    }    ...}
```

`AttendanceService.scanAttendance()` calls `GymGuard.requireStaffGym(scanner)` before doing anything else, so a MEMBER-role JWT is rejected with HTTP 403 before any database write occurs. Symmetrically, `scanGymQrAsMember()` explicitly checks that the caller's role is MEMBER.

5.4.2 Business-Rule Validation

- **Gym scoping:** `scanAttendance()` compares `member.getGym().getId()` to the scanning staff member's gym and throws `IllegalArgumentException` if they differ, preventing cross-branch check-ins.
- **Account status:** an inactive member account (`isActive() == false`) is rejected on both the QR-issuance and scan paths.
- **Subscription gating:** `MembershipStatusUtil.allowsAttendance(status)` permits only ACTIVE or EXPIRING_SOON memberships to start a new check-in; a member who is already checked in is still allowed to check out even if their membership lapsed mid-visit.
- **Malformed QR payloads:** `parseMemberIdFromQr()/parseGymIdFromQr()` validate the "GYMTRACK|TYPE|id" shape and numeric ID, throwing `IllegalArgumentException` ("Invalid member QR format." / "Invalid member QR code.") for anything else, which the controllers map to HTTP 400.

5.4.3 Controller-Level Error Mapping

Both `QrController` and `AttendanceController` use an identical try/catch pattern so business exceptions become predictable HTTP responses for the frontend to branch on:

```
try {    return ResponseEntity.ok(attendanceService.scanAttendance(request.getQrData(),
authentication.getName()));} catch (IllegalArgumentException ex) {    return
ResponseEntity.badRequest().body(ex.getMessage());    // 400} catch (SecurityException ex) {
return ResponseEntity.status(403).body(ex.getMessage());    // 403}
```

5.5 Data Storage and Display

Attendance events are persisted in the `attendance_logs` table via the `AttendanceLog` JPA entity (`user_id` foreign key, `check_in_time`, `check_out_time`). A check-in inserts a new row; a check-out updates the `check_out_time` of the most recent open row for that user, found via

`AttendanceLogRepository.findTopByUserIdOrderByCheckInTimeDesc()`. Membership status is not duplicated onto the log — it is looked up live from the `memberships` table at scan time and at read time, so historical logs always reflect the plan/status resolved at query time rather than a stale snapshot.

On the display side, `GET /api/attendance/me` and `GET /api/attendance/gym` return `AttendanceLogResponse` lists that are rendered by `AttendanceLogTable.jsx` / `AttendanceHistory.jsx` (web) and their Kotlin equivalents (mobile), with the gym-scoped endpoint additionally supporting free-text and date filtering for staff/admin.

5.6 Known Limitation Identified During Review

The QR payload for a member ("GYMTRACK|USER|<id>") contains no expiry, nonce, or signature. It is therefore a static, indefinitely reusable identifier: whoever presents that exact string — whether the member themselves, a screenshot, or a shared image — is treated as that member for check-in purposes, as long as a STAFF/ADMIN account performs the scan. Staff/admin-only scanning is correctly enforced (Section 5.4.1), but there is no mechanism confirming the physical presence of the actual member behind the QR. A future improvement would be a rotating/time-boxed token (e.g., HMAC-signed with a short validity window) and/or displaying the member's name/photo on the scan-confirmation screen for staff to visually verify before finalizing the check-in.

6. Commit History Table

Each row corresponds to a distinct implementation task for this feature, with its associated commit in the GymTrack repository. Hashes link directly to the commit on GitHub.

Task / Functional Requirement	Description	GitHub Commit
Introduce QR + Attendance Backend	Added the initial QrAttendanceService, AttendanceController, QrController, AttendanceLog entity/repository on the backend, plus the first mobile QR scanner overlay and member QR card (bundled into a broader mobile-auth commit).	d47a6ce
Add Membership Status Gating for Attendance	Introduced the Membership entity/service, MembershipStatusUtil, and expanded QrAttendanceService (~255 line change) to require an active subscription before issuing a member QR or allowing check-in; added gym QR card, membership card, and attendance history UI on web and mobile.	284c6d0
Refactor Attendance Feature into Vertical Slice	Restructured attendance/QR code from a layered (controller/service/dto/entity packages) architecture into a single feature/attendance package (AttendanceService, AttendanceController, QrController, GymGuard usage), matching the same refactor applied to auth, plans, members, membership, and payments.	7740660
Fix GYM Entity Bugs Affecting Scan Validation	Fixed bugs in the Gym/User entity relationship and AuthResponse that affected gym-scoping checks used during attendance scanning.	44e7acb
Fix QR Code Bug and Update Scan UI	Fixed a bug in the QR/attendance flow and updated the mobile dashboard and web navigation UI around scanning and status displays.	be3010b

Note: The five commits above are the ones that directly touch the attendance/QR module.

6.1 Full Repository Commit History

24 commits on the `main` branch, oldest to newest, each linking to the commit on GitHub. Rows shaded amber are the same five rows shown in the table above.

Date	Commit	Message
2026-07-01	b9449f6	Initial Commit
2026-07-01	ccb74f8	second commit
2026-07-01	aa881cd	fix some bugs

Date	Commit	Message
2026-07-01	2b11064	Create README.md
2026-07-01	6b9b568	added dashboard
2026-07-02	38ec787	Initializes the mobile
2026-07-02	d47a6ce	Added the mobile login and register
2026-07-04	284c6d0	updates
2026-07-05	0e3151b	layout updates and ui
2026-07-07	e87ed8d	refactor(auth): vertical slice for user registration
2026-07-07	544bab8	refactor(auth): vertical slice for user login
2026-07-07	3407f40	refactor(auth): vertical slice for admin staff creation
2026-07-07	672bfbe	refactor(plans): vertical slice for plan management
2026-07-07	887c770	refactor(members): vertical slice for member management
2026-07-07	8ac7024	refactor(membership): vertical slice for membership status
2026-07-07	f7f26f7	refactor(payments): vertical slice for PayMongo checkout
2026-07-07	7740660	refactor(attendance): vertical slice for QR check-in/out
2026-07-07	61cd3ea	refactor(dashboard): vertical slice for admin KPIs
2026-07-07	65f2c36	refactor(staff): vertical slice for staff accounts
2026-07-07	29af8a7	REM
2026-07-07	4088f3b	Create Lim_LouisFrancis_Phase2.pdf
2026-07-08	44e7acb	Fixes some bugs regarding on GYM entity
2026-07-09	6f58dfb	Fixes the bug where the sign in button dosen't work
2026-07-09	be3010b	Fixes the qr code bug and update the UI